Champlain
COLLEGE SAINT-LAMBERT

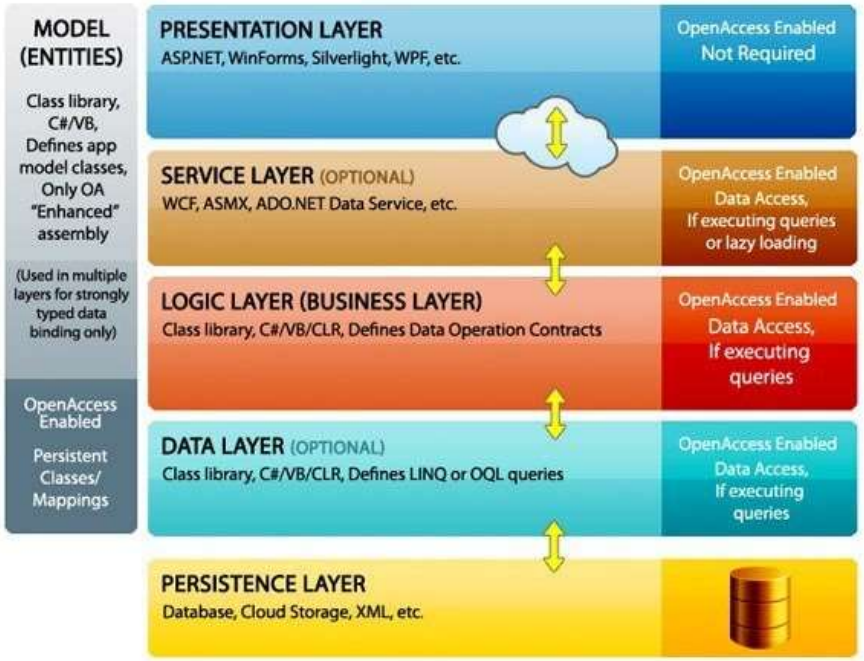
420-N55-LA

IoT Design and Prototyping

What is N-Tier Design?

N-Tier Architecture - Layers

- An N-tier architecture divides an application into logical layers and physical tiers.
- N Tier uses:
  - E-Commerce commercial applications.
  - CRM's
  - Academic applications.
  - Intranets.
- N Tier NON uses:
  - If creating a simple web page – no need for this complex structure!



Analogy

No need to organize

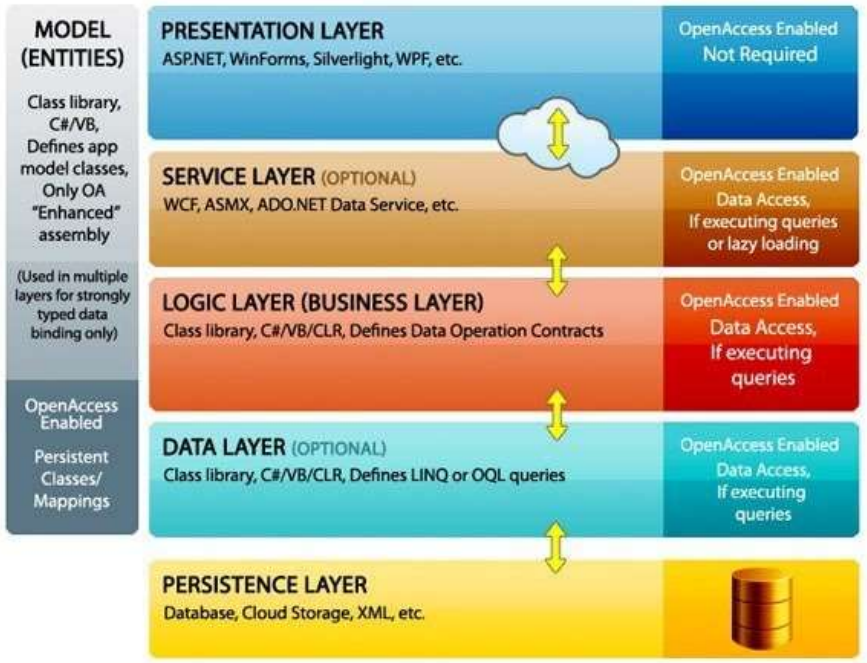
Organize into N-tier much better

BEFORE

AFTER

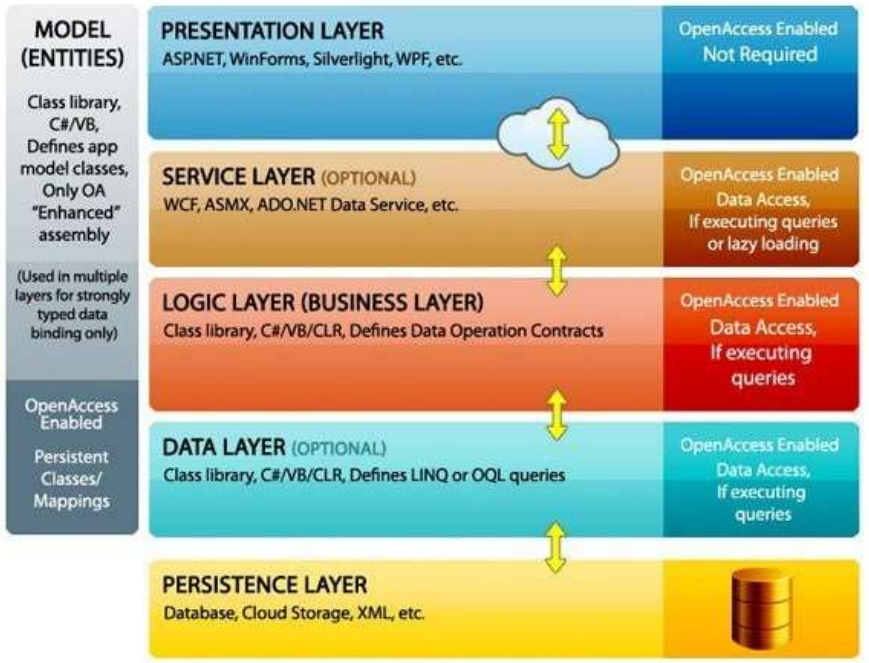
Benefits

- Separation of Duty
- Low/loose coupling
- High cohesion
- Mixing of technology
- Modular
- Easy maintenance
- No dependence on technology, operating system, server type.



Layer

- Layers are a way to separate responsibilities and manage dependencies. Each layer has a specific responsibility or functionality.
- A higher layer can use services in a lower layer, but not the other way around.
- Data must flow to/from each layer.

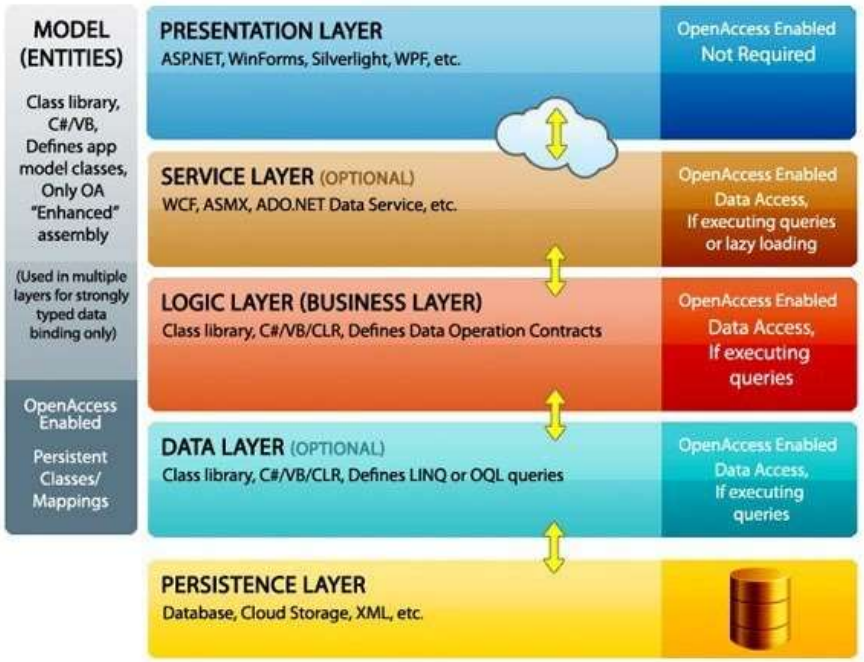


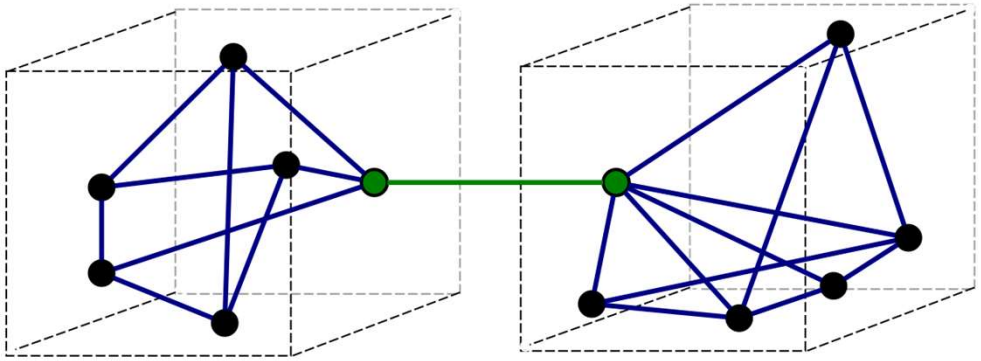
Tier

- Tiers are physically separated, running on separate machines.
- A tier can call to another tier directly, or use asynchronous messaging (message queue).
- Although each layer might be hosted in its own tier, that's not required. Several layers might be hosted on the same tier.
- Conversely, ALL layers can also be hosted by one tier.
- Physically separating the tiers improves scalability and resiliency, but also adds latency from the additional network communication.

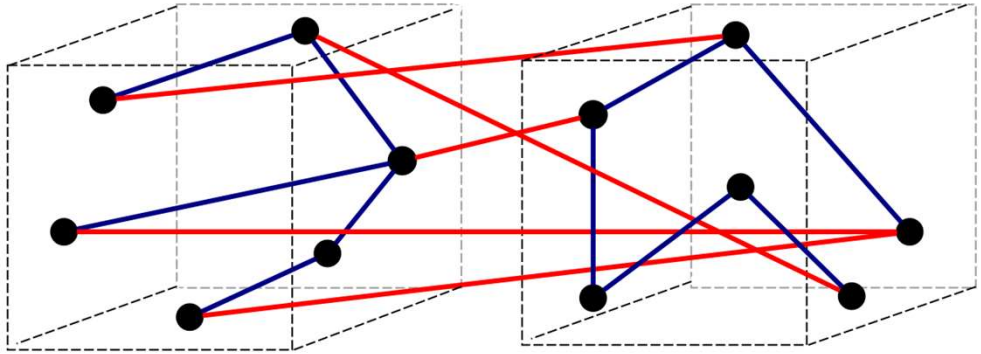
Open and Close Tier

- An N-tier application can have a **closed** layer architecture or an **open** layer architecture:
  - In a closed layer architecture, a layer can only call the next layer immediately down.
  - In an open layer architecture, a layer can call any of the layers below it.





a) Good (loose coupling, high cohesion)



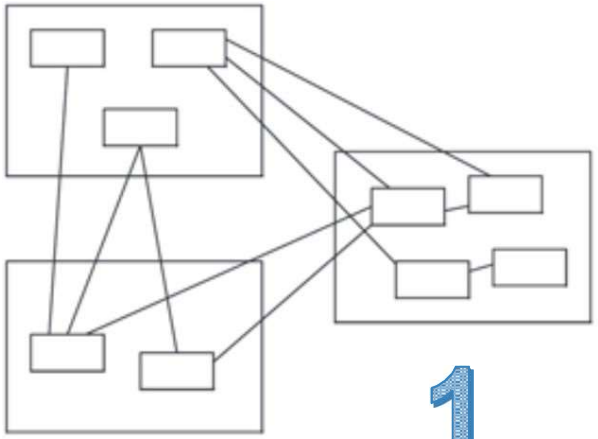
b) Bad (high coupling, low cohesion)

How do you implement loose coupling?

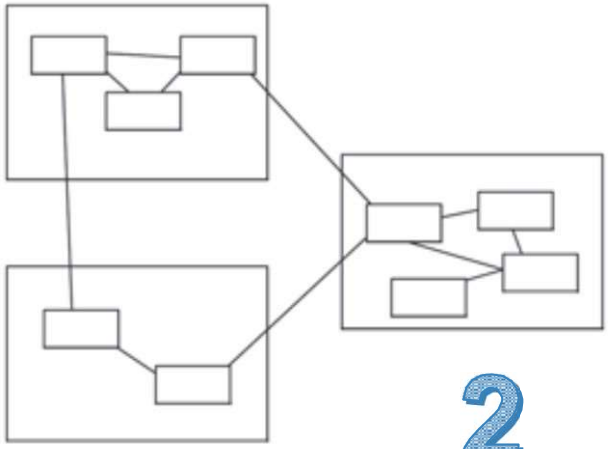
Each function shouldn't be dependent on another function in another layer.

How about high cohesion?

Put similar functions in the same layer. Required components are self-contained.



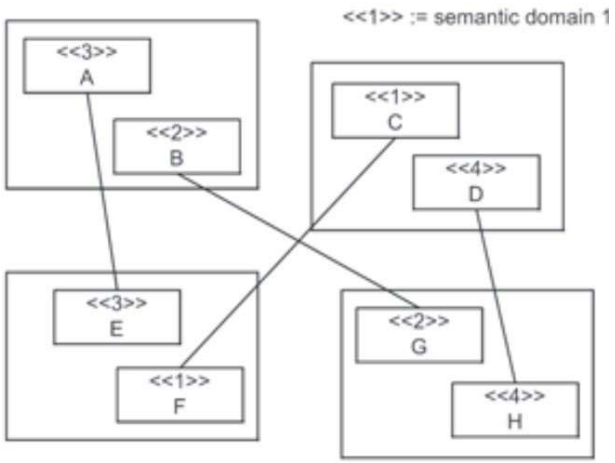
1



2

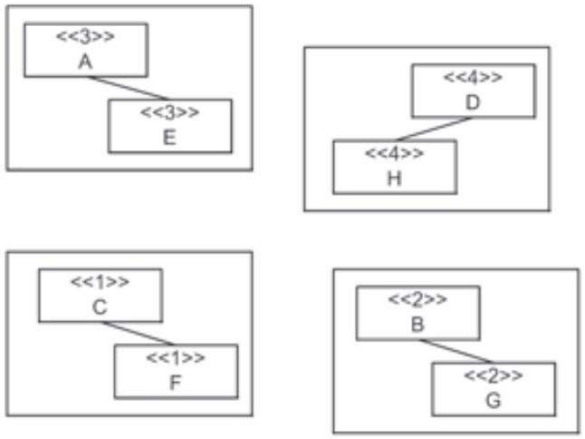
Strong or loose coupling?

What do you think?



1

2



High or Low Cohesion?

For the class to answer...

Disadvantages?

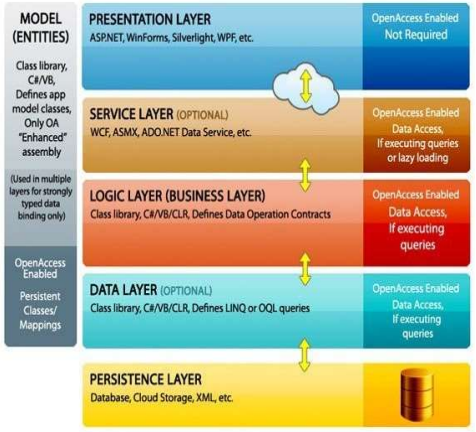
- Longer to develop.
- More complex to test.
- Requires more skilled programmers.

Presentation Layer (client machine)

- Can be implemented as a simple web-page using JavaScript/React/Angular.
- Some layer technology like Angular-JS uses MVC to organize its structure, in the presentation layer.
- An Android application may be considered a form of a presentation layer (or client).
- Older tech like JSP pages, PHP pages – can all be clients.
- Non-web clients like a windows application – can also be considered a client.

Service Layer

- Required to interface technologies of different types and achieve low coupling.
- Service layers are web services which listen on the internet, behind an IP address and return information when called.
- Think of a service as a class with methods, you call the method and it returns (a value/values) to you.
- Services are usually RESTFUL. RESTFUL services follow a pattern to get data, insert data, update data, delete data.
- Restful services usually use JSON to communicate data between client and service.
- Very little validation performed here.
- Very important – must implement security here (HTTPS, authentication, cookies, tickets).



Service Layer Technologies or frameworks

- WebAPI (Microsoft)
- Jersey, Spring, Spring Boot (Java)
- Laravel, Lumen (PHP)
- Express, Slim, Restify (Javascript, Node JS)
- Django REST, Flask Restful, Falcon (Python)

REpresentational State Transfer (REST) is a software architectural style that was created to guide the design and development of the architecture for the World Wide Web. REST defines a set of constraints for how the architecture of an Internet-scale distributed hypermedia system, such as the Web, should behave.

Logic Layer (or Business Layer)

- This contains all your Logic methods and usually the programs which run your system.
- Here we perform calculations and other logic if necessary.
- Often the entities (Classes) are stored here.
- This layer contains classes, enums, and the necessary methods.
- The Logic layer performs validation (check for existing record, make sure the data sent is valid)
- The Logic layer checks for logical authorization (must be allowed to get the data).

Data Access Layer (Persistence)

- Sometimes separated.
  - Data Access: Entity framework, Entities.
  - Data persistence: Simple logic that reads/writes to a database.
- More often, it's just one layer called "Data Access Layer".
- Less validation if any happens here.
- Errors must be handled properly.
- Should be written in a decoupled way.
- Here you find raw Structured Query Language (SQL) commands like "insert/update" data.

Data Layer Languages and Frameworks

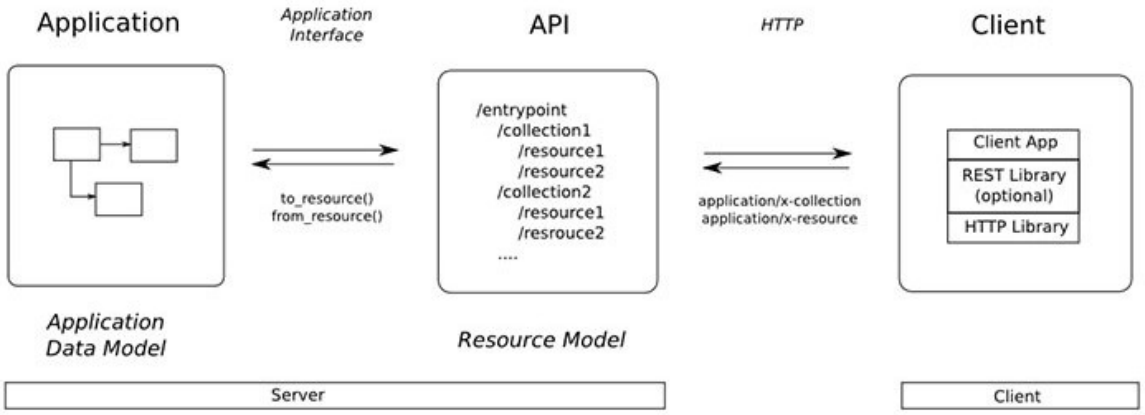
- Popular Data languages and Frameworks
  - Structured Query Language (SQL)
  - Hibernate
  - Entity Framework
- Popular Databases:
  - MySQL
  - MS SQL Server
  - SQL Lite
  - MongoDB
  - Postgres DB

Frontend/Backend/Full Stack?

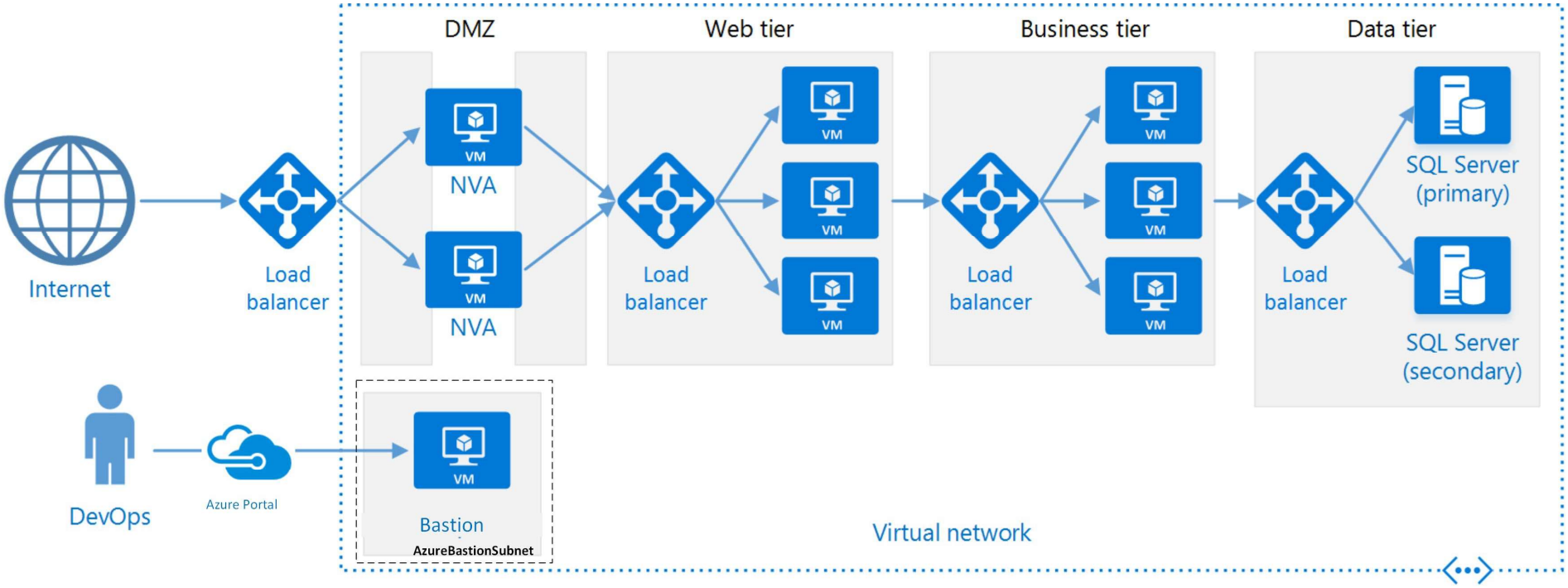
- Front-end technologies include the web pages, android screens and sometimes the **client** apps run on a computer.
- Front end programmers usually are heavily versed in Javascript and aren't used to using **typed languages like Java and C#**.
- Other popular front-end languages and markup code include;
  - Typescript
  - HTML
  - CSS
  - Bootstrap
- Frontend development depends highly on **Agile** development.

Frontend or Client

- A client usually has a class (or library) in it that helps it create REST requests. This library should be separate from the code and easy to replace.

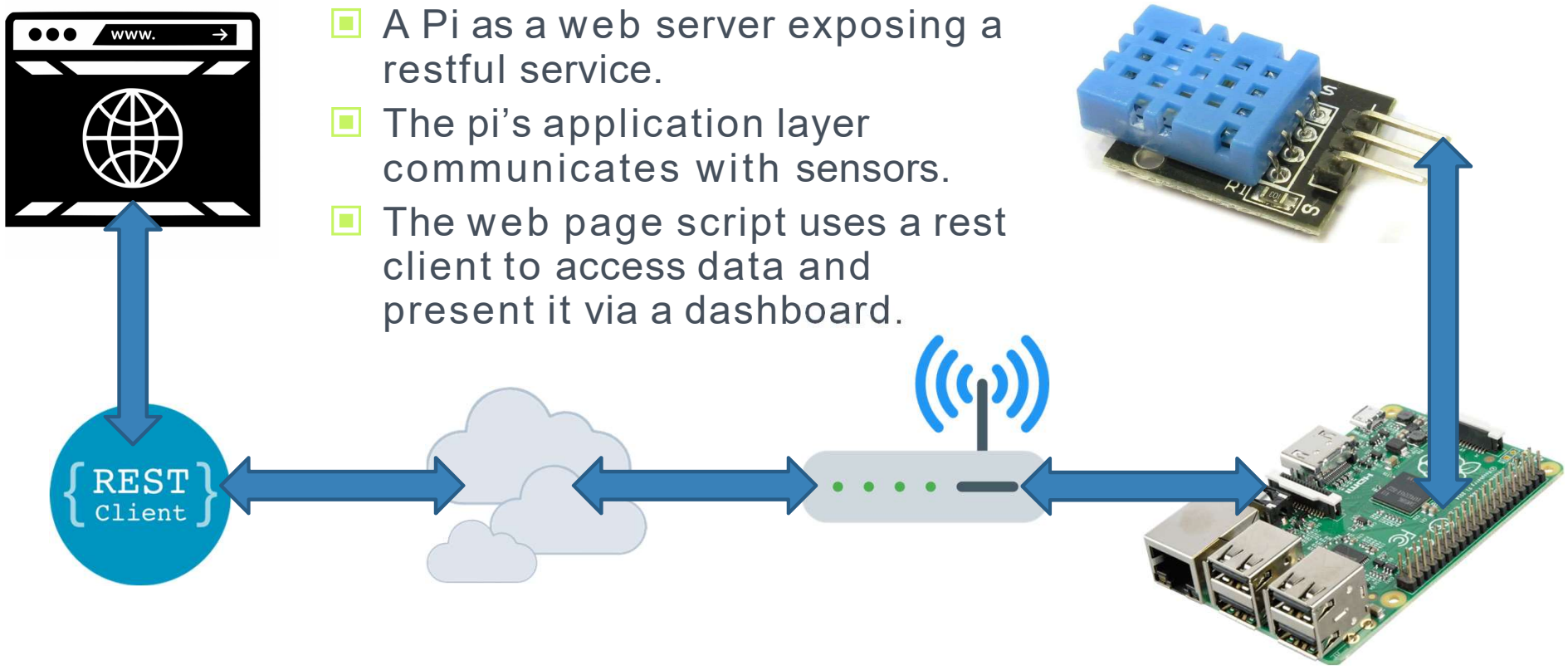


Recommended N-tier Architecture for Large Enterprise



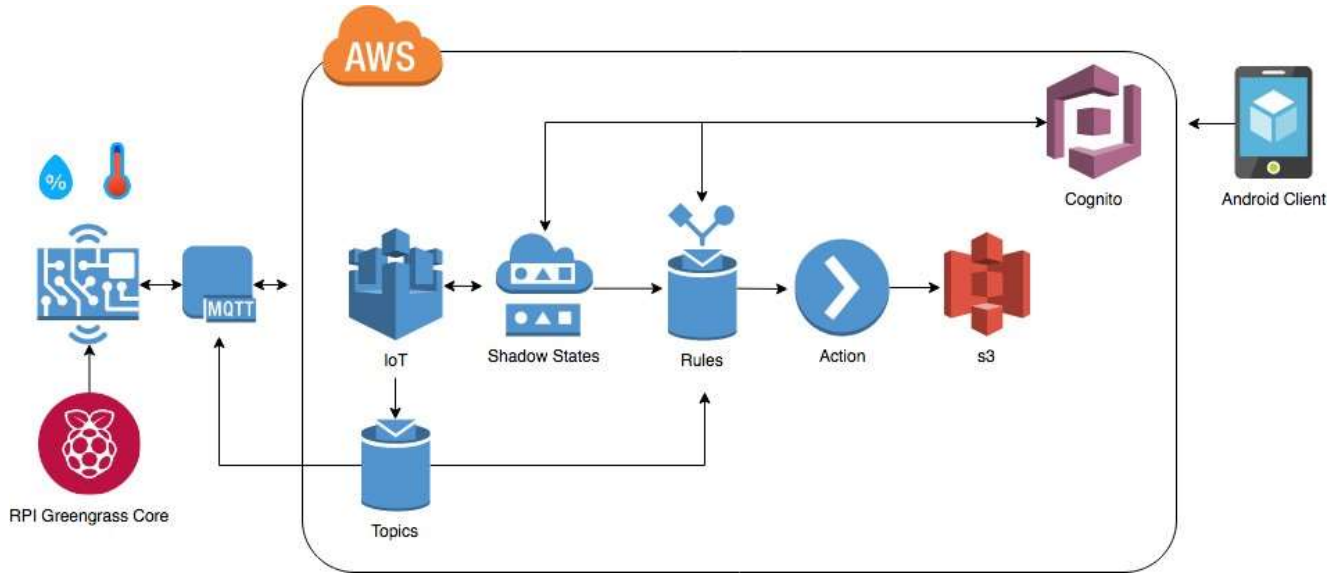
A Sample Setup (Pi Server)

- A Pi as a web server exposing a restful service.
- The pi's application layer communicates with sensors.
- The web page script uses a rest client to access data and present it via a dashboard.



AWS Setup

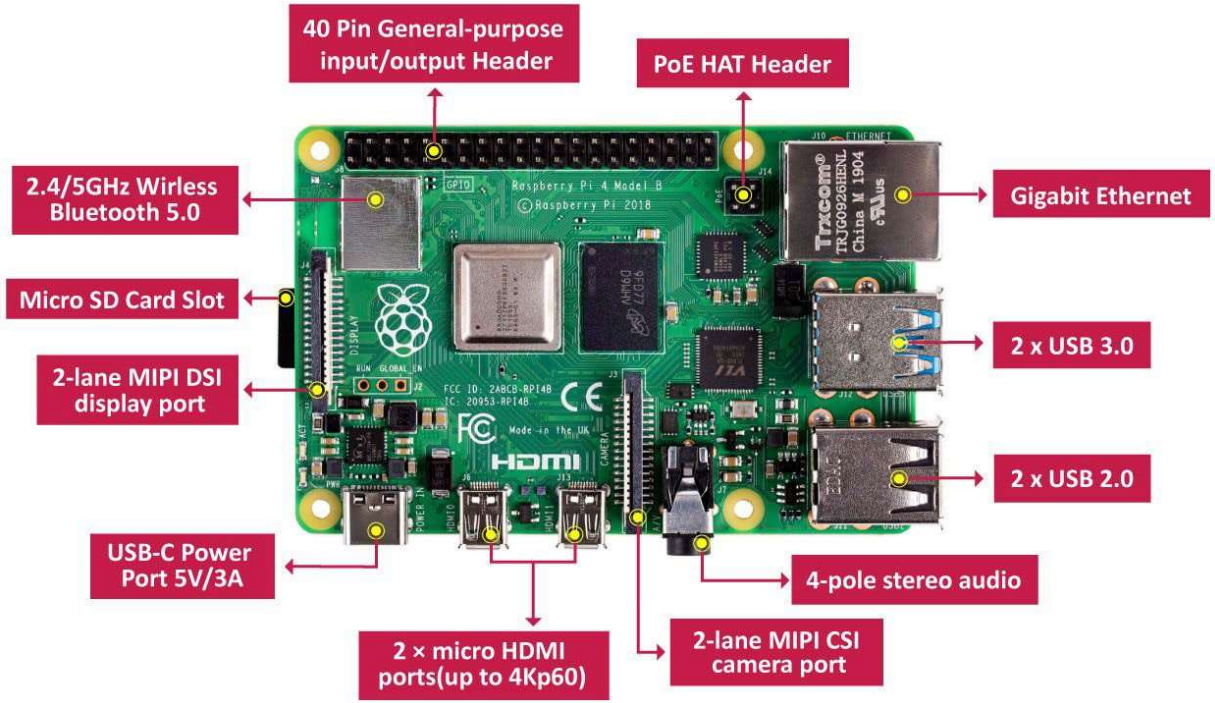
- The following setup publishes data to Amazon Web Services, which in turn has a dashboard which can be displayed to the public.
- Data is stored in Amazon's s3 service.
- This is not considered an "N-Tier" application. But does exhibit qualities of it.



Intro to Raspberry Pi and Raspbian OS

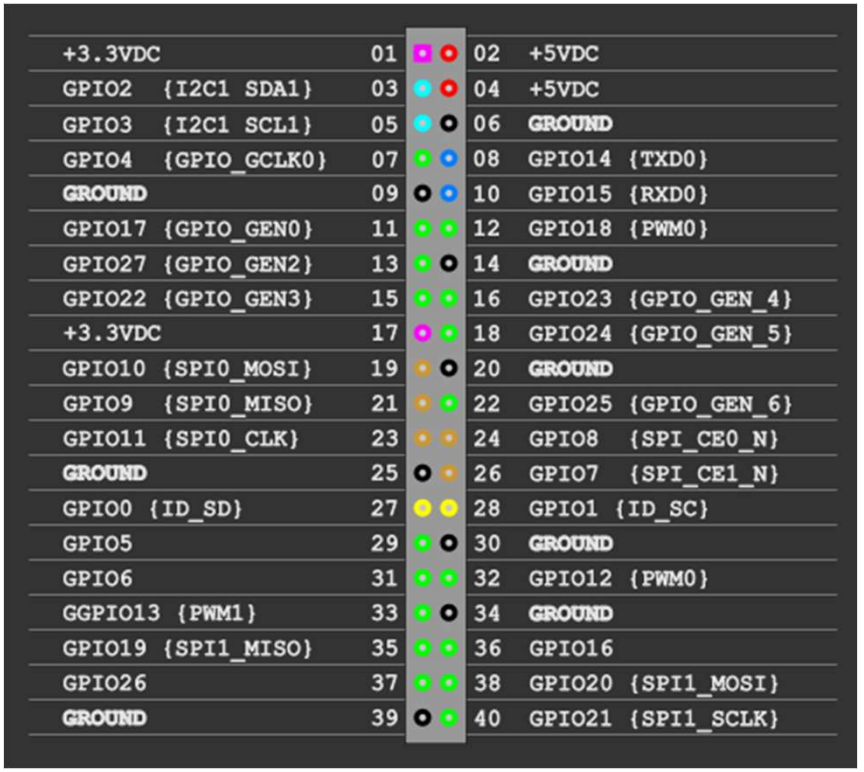
Eg: Your shopping list ☺

Anatomy of a Pi



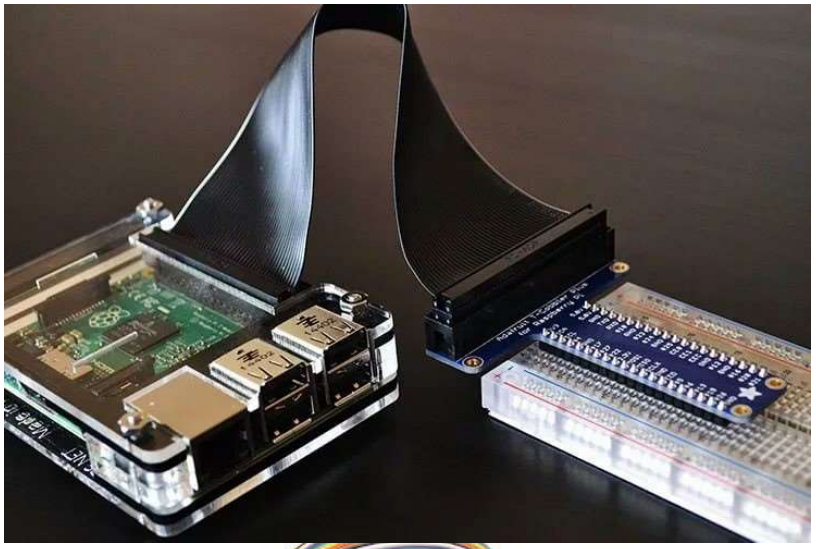
GPIO Port

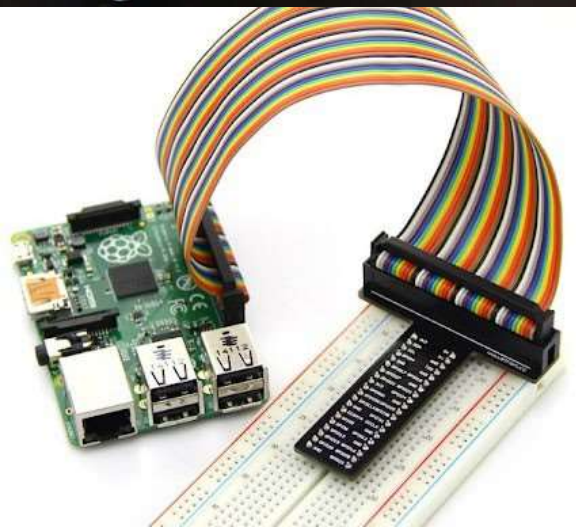
- Make note of:
  - Ground
  - +5v, +3.3v
  - PWM0, PWM1
    - Pulse width modulation.
  - TXD, RXD
    - Serial transmit and receive.

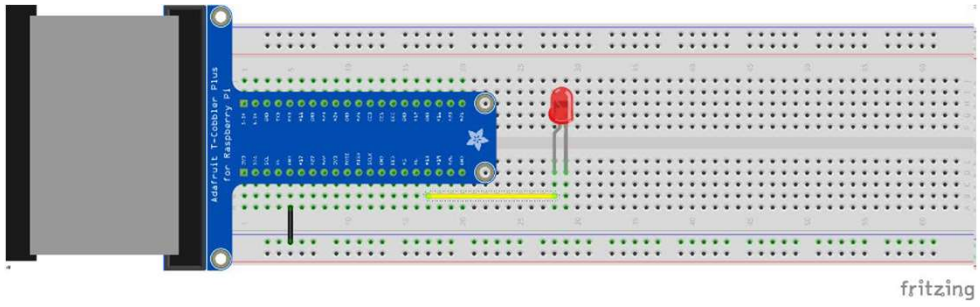


GPIO Extension

- This allows you to hook up a breadboard externally to the Pi.
- Many cases allow you to keep the Pi in the case and have a ribbon cable coming out a slot.
- The rest, you will see, will be almost like what we did with the Arduino.







Champlain

COLLEGE SAINT-LAMBERT